

CS 342300 Operating Systems - Programming Project

Checkpoint 2

Preemptive Multithreading Package

Name	Bryan Gilbert
Student ID	_____
Submitted files	testpreempt.c, preemptive.c, preemptive.h, Makefile, ppc2.pdf

1. Code Overview

This checkpoint changes the Checkpoint 1 cooperative threading package into a preemptive threading package. The Producer and Consumer no longer call ThreadYield(). Timer0 generates interrupts, and timer0_ISR jumps to myTimer0Handler, which saves the current thread state, chooses the next runnable thread using round-robin scheduling, restores that thread state, and returns with RETI.

Shared variables sharedChar and bufferFull are protected by disabling interrupts with EA = 0 before the critical access and enabling interrupts again with EA = 1 afterward. Consumer uses TMOD |= 0x20 so Timer1 can be configured for UART without destroying the Timer0 settings used for preemption.

SAVESTATE pushes ACC, B, DPL, DPH, and PSW. RESTORESTATE pops them in reverse order. The saved stack pointer is stored in fixed variables savedSP0 to savedSP3, avoiding array indexing inside the interrupt handler because SDCC may use R0/R1 for indexing.

2. Compilation Screenshots

Run these commands in the folder containing the submitted files:

```
$ make clean
$ make
```

Screenshot: make clean

Paste screenshot of make clean output here

Screenshot: make

Paste screenshot of make output here

Expected generated files include testpreempt.hex, testpreempt.map, testpreempt.asm, testpreempt.rel, and related SDCC output files. testpreempt.hex is loaded into EdSim51. testpreempt.map is used to find symbol addresses for breakpoints.

3. Symbol Addresses from testpreempt.map

After compiling, look up the following symbols in testpreempt.map and fill in their addresses.

Symbol	Address from testpreempt.map	Purpose
_Bootstrap		Startup jumps here to create the main thread and start Timer0.
_ThreadCreate		Creates a fake initial stack for a new thread.
_Producer		Producer loop writes A-Z into the shared buffer.
_Consumer		Consumer loop reads the shared buffer and writes to UART.
_timer0_ISR		Timer0 interrupt entry in testpreempt.c.
_myTimer0Handler		Actual preemptive context switch handler.
_sharedChar		Single-byte shared buffer at internal RAM 0x30.
_bufferFull		Flag at internal RAM 0x31.

4. EdSim51 Screenshots and Explanations

4.1 Before Bootstrap calls ThreadCreate(main)

Paste screenshot before ThreadCreate(main) in Bootstrap

Explanation: Before the first ThreadCreate call, the bootstrap code has initialized the thread bitmap to 0 and Timer0 is configured but global interrupts are still disabled. ThreadCreate(main) allocates thread ID 0, creates a fake stack starting at 0x3F, pushes the return address for main, pushes zero initial register values, pushes PSW = 0x00 for register bank 0, and stores the new SP in savedSP0.

4.2 Before main calls ThreadCreate(Producer)

Paste screenshot before ThreadCreate(Producer) in main

Explanation: At this point thread 0 is running main/Consumer. ThreadCreate(Producer) allocates thread ID 1 because bit 0 is already used. It creates Producer stack space starting at 0x4F, pushes Producer as the fake return address, pushes zero register values, pushes PSW = 0x08 to select register bank 1, and stores the new SP in savedSP1.

4.3 Producer running

Paste screenshot when Producer is running

Explanation: Producer is running when execution is inside the `_Producer` symbol range from the map file. Another clue is that `sharedChar` changes through A, B, C, ..., Z and `bufferFull` becomes 1 after Producer writes into the buffer.

4.4 Consumer running

Paste screenshot when Consumer is running

Explanation: Consumer is running when execution is inside the `_Consumer` symbol range. It reads `sharedChar` into `consumedChar`, clears `bufferFull` to 0, writes `consumedChar` to `SBUF`, and polls `TI` until the UART transmission finishes.

4.5 Timer0 interrupt is triggering regularly

Paste screenshot showing repeated Timer0 ISR / myTimer0Handler execution

Explanation: Timer0 is triggering regularly if the program repeatedly enters timer0_ISR and jumps to myTimer0Handler without any explicit ThreadYield calls in Producer or Consumer. In EdSim51, this can be observed by setting a breakpoint at _timer0_ISR or _myTimer0Handler and seeing that execution repeatedly stops there while the output continues to alternate between Producer and Consumer activity.

5. Notes About Correctness

- The Producer and Consumer do not call ThreadYield; switching is caused by Timer0.
- ThreadCreate, ThreadYield, and ThreadExit are protected from Timer0 preemption by disabling EA during their critical work.
- myTimer0Handler ends with RETI instead of RET because it is entered from an interrupt.
- The scheduler and state save/restore code avoid R0-R7 in the interrupt path to reduce SDCC register corruption bugs.
- TMOD uses |= 0x20 in Consumer so UART Timer1 configuration does not erase Timer0 settings.